

Appendix: Artifact Description

PDSW'26 — WarpX / ADIOS2 I/O benchmarking study

Artifact Description (AD)

I. Overview of Contributions and Artifacts

A. Paper's Main Contributions

- C_1 Balanced-workload write-path characterization on Frontier and Perlmutter (weak scaling; multi-TB/s, including Blosc2).
- C_2 ADIOS2 aggregation (TwoLevelShm, EveryoneWritesSerial, DataSizeBased) under extreme writer-side skew.
- C_3 FlattenSteps for lab-frame BTB snapshots from non-contiguous boosted-frame data at negligible write-time overhead.
- C_4 SPAN and AsyncWrite for buffering and compute/I/O overlap under BTB imbalance.
- C_5 Practical ADIOS2 configuration guidelines for WarpX-like PIC codes.

B. Computational Artifacts

A_1 <https://github.com/guj/artifacts26> — Frontier build/env scripts, WarpX inputs, PerformanceRun and repeat/ jobs, log timer extractor, and result CSVs. (Optional Zenodo DOI may be added after acceptance.)

Art.	Contrib.	Related paper elements
A_1	C_1 – C_5	Table I; Fig. 2 (Perlmutter 3D); Figs. 3–6 (BTB); small- N mean $\pm$ stdev (§II-B, §III-C, §IV-B); §VI-B recommendations

II. Artifact Identification

A. Computational Artifact A_1

Relation To Contributions

A_1 packages inputs, ADIOS2 parameterizations, repeat/ scripts, the log timer extractor, and CSVs for Table I and Figs. 2–6. It supports replotting from CSVs and, where allocations permit, re-execution of the WarpX $\rightarrow$ openPMD-api $\rightarrow$ ADIOS2 write path.

Expected Results

- Table I (Frontier 3D; EWS/TLS $\pm$ Blosc2): relative ordering is the primary claim; absolute TB/s varies with Lustre contention.
- Fig. 2 (Perlmutter 3D): TLS above EWS; aggregate >1 TB/s at 512 nodes.
- BTB: FlattenSteps overhead negligible vs. EWS; SPAN+AsyncWrite highest among tested configs; Data-SizeBased preferred when subfile count is constrained under skew.

Expected Reproduction Time (in Minutes)

- Setup: 30–120 min (build stack or load site modules).
- Execution: ≈ 1 –2 min compute at 8 nodes (Slurm 5–10 min); queue wait is site-dependent. Full scale (2048 Frontier / 512 Perlmutter nodes) needs a leadership allocation and terabytes of scratch (Fig. 2: a 128-node Perlmutter 3D step is already ≈ 10 TB).
- Analysis: <15 min to import CSVs and replot in Google Sheets.

Artifact Setup (incl. Inputs)

Hardware:

- Frontier (ORNL; primary documented platform): HPE Cray EX; AMD MI250X; Lustre/ClusterStor ORION (≈ 5 TB/s peak write); weak scaling to 2048 nodes; 8 CPU cores/node for I/O. Setup and job scripts in the repo target Frontier.
- Perlmutter (NERSC): GPU nodes; all-flash Lustre (>5 TB/s); weak scaling to 512 nodes (scratch-quota limited); 4 CPU cores/node for I/O. Same WarpX $\rightarrow$ openPMD-api $\rightarrow$ ADIOS2 stack; only modules, account/QOS, and scratch paths differ. Example: `warpx_tests/PerformanceRun/BTD/N8/perlmutter.sh`. Node-local NVMe burst buffers were explored for BTB staging but are not required to regenerate the primary CSV/plot artifacts.

Software:

- WarpX 25.12 (<https://github.com/BLAST-WarpX/warpx>)
- openPMD-api bundled with that WarpX build (<https://github.com/openPMD/openPMD-api>)
- ADIOS2 2.11, BPFFile / BP5 (<https://github.com/ornladios/ADIOS2>)
- Frontier modules (scripts/frontier/bash.setup.env): `cmake/3.30.5`, `craype-accel-amd-gfx90a`, `rocm`, `cray-mpich/8.1.31`, `cce/18.0.1`, `hdf5/1.14.5-mpi` (optional `ccache`, `ninja`)
- Lossless Blosc2 via ADIOS2 (zstd level 1, bit-shuffle, 2048-byte threshold, 6 threads/rank)
- Python 3.6+ for scripts/frontier/extract_time_3.6.py; Google Sheets to import results/*.csv (no matplotlib drivers)

Datasets / Inputs: No external observational datasets. Decks and Slurm scripts live in `warpx_tests/`; three-trial scripts in `repeat/`. Per-scale decks: `input.n${NNODES}`. Scripts under `repeat/` resolve EXE, bpls, and inputs_3d relative to `warpx_tests/` (set `#SBATCH -A YOUR_PROJECT`).

- `inputs_3d/opmd/regular/`, `regular_blosc/`, `regular_h5/`: 3D Cartesian LWFA (≈ 33 M cells/node; ≈ 2.5 GB/node/step).

- inputs_3d/opmd/btd_*/, btd_flatten*/: BTD (skew + out-of-order; FlattenSteps variants).
- PerformanceRun/3D/, PerformanceRun/BTD/: Frontier jobs (plus BTD/.../perlmutter.sh); SST_BTDD/ is exploratory.
- repeat/: same pattern, by config (bp/, bp_async/, bp_flatten/, h5/); submit each script three times for $N=8-32$ (repeat/README.md).

Paper numbers are in results/: frontier_3d_throughput.csv (Table I); 3D-perlmutter.csv (Fig. 2); fig_btd_a_flatten_frontier.csv (Fig. 3); fig_btd_b_span_async_frontier.csv (Fig. 4); fig_btd_c_agg_perlmutter.csv (Fig. 5); fig_btd_d_subfiles_perlmutter.csv (Fig. 6); repeat_runs_raw.csv, repeat_summary.csv (mean±stdev). See warpx_tests_INDEX.md, repeat/README.md, results/README.md.

Installation and Deployment: Frontier is the documented host. Choose scratch WORKDIR (no hard-coded user path) and run:

- 1) export WORKDIR=
/lustre/orion/<project>/scratch/<user>/<exp>
- 2) source scripts/frontier/bash.setup.env (modules; ADIOS2_DIR; AMREX_AMD_ARCH=gfx90a; GPU-aware MPI).
- 3) ./scripts/frontier/build.setup builds ADIOS2 2.11 and WarpX 25.12 under \$WORKDIR/binary/ with bundled openPMD-api (WarpX_openpmd_internal=ON). Override with ADIOS2_TAG, WARPX_TAG.
- 4) In warpx_tests/, symlink EXE → WarpX and bpls → ADIOS2 bpls; set #SBATCH -A.
- 5) Submit, e.g. sbatch PerformanceRun/3D/N1/frontier_blosc.sh or sbatch PerformanceRun/BTD/N8/frontier.sh. Small- N repeats: from repeat/..., submit each script three times.
- 6) From the parent of each job_id/ directory: python3 scripts/frontier/extract_time_3.6.py job_id (bare id, no slash) → TotalTime, IOTime. Option 1 I/O time is $\text{TotalTime}_{\text{I/O}} - \text{TotalTime}_{\text{nullcore}}$ when nullcore runs exist; DataSize is the observed on-disk output size.
- 7) Aggregate into results/*.csv; replot in Google Sheets.

Perlmutter: same stack and inputs_3d/; start from PerformanceRun/BTD/N8/perlmutter.sh and adjust account/QOS, modules, and scratch.

Artifact Execution

T_1 build/load → T_2 run WarpX → T_3 extract_time_3.6.py plus file sizes into CSV → T_4 Google Sheets. Parameters: node count, aggregation, NumSubFiles, FlattenSteps, SPAN, AsyncWrite, Blosc2. Frontier $N=8-32$ configs were repeated three times (repeat/); larger scales and Blosc are single-trial, with reliability via cross-scale consistency (§III-C).

Artifact Analysis (incl. Outputs)

CSV throughput vs. nodes plus small- N mean±stdev; plots of Figs. 2–6 from those CSVs. Compare to Table I and Figs. 2–6 on relative trends; document filesystem contention as in §VI-C.